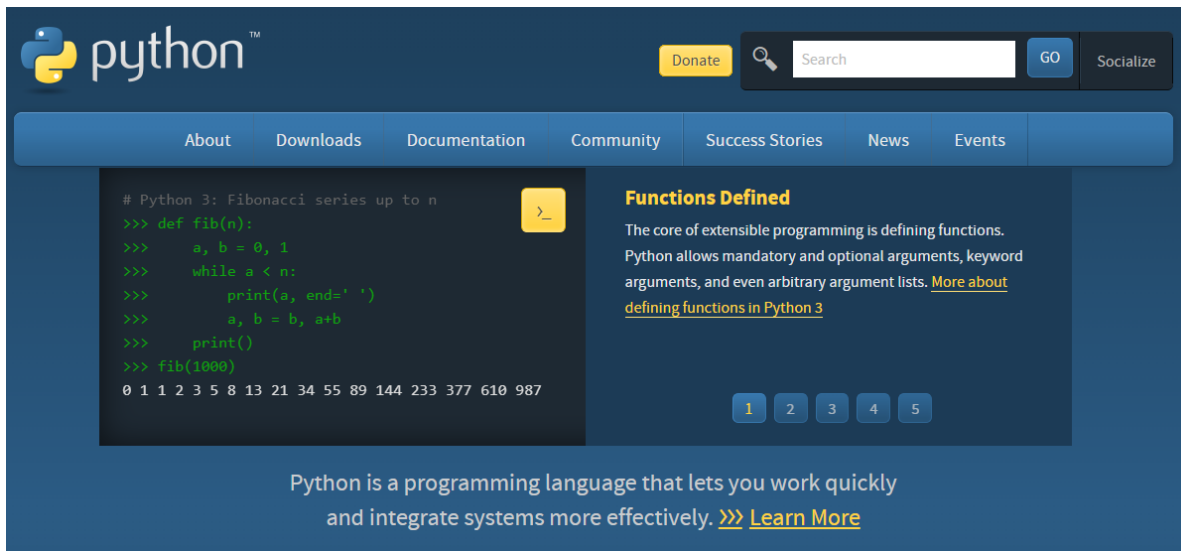
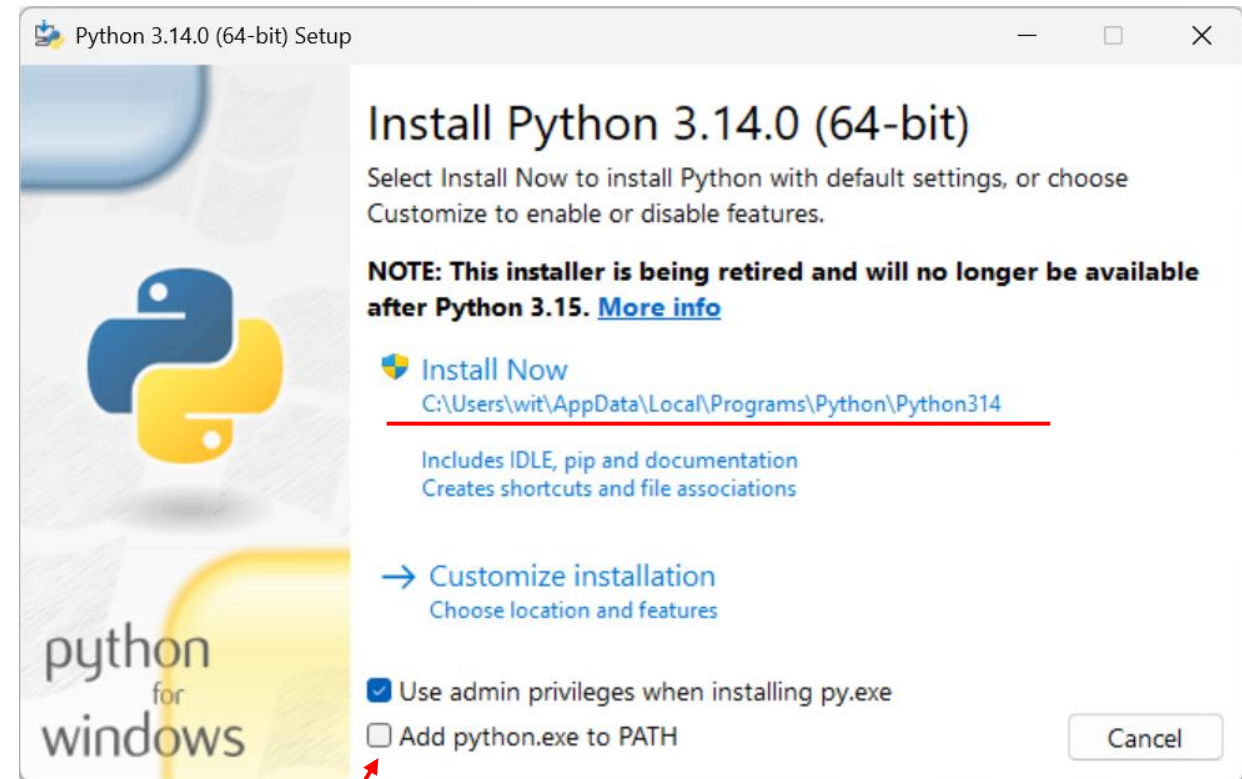




- python3.x.y (np. 3.13.7)
- nie instalujemy najnowszej wersji, np. 3.14.y (y=0, 1) chyba, że w celach edukacyjnych



tak

The image shows a screenshot of the Visual Studio Code (VS Code) interface. At the top, the menu bar includes File, Edit, Selection, and View. The GitHub Copilot menu is open, displaying options like 'Status: Ready', 'GitHub Copilot Chat', 'Disable Completions', 'Edit Keyboard Shortcuts...', 'Edit Settings...', 'Show Diagnostics...', 'Open Logs...', 'View Copilot Documentation...', and 'View Copilot Forum...'. A red arrow points to the 'terminal u dołu' (terminal at the bottom) icon in the top right corner. Another red arrow points to the 'Ctrl,' shortcut in the settings for 'Editor: Mouse Wheel Zoom'. Below the Copilot menu, the settings for 'Editor: Mouse Wheel Zoom' are shown, with a red circle around the 'Zoom the font of the editor when using mouse wheel and holding ctrl.' checkbox. In the center, the text 'PEP – Python Enhancement Proposals' is followed by the URL <https://peps.python.org/pep-0000/>. Below this, a list of command shortcuts is shown, with 'Show All Commands' (Ctrl + Shift + P) circled in red. A large white arrow points from the 'Show All Commands' section down to the text 'terminal u dołu: Ctrl `'. On the right side, the word 'COPILOT' is written above a large red arrow pointing down to the Copilot icon in the bottom right corner of the VS Code window.

terminal u dołu

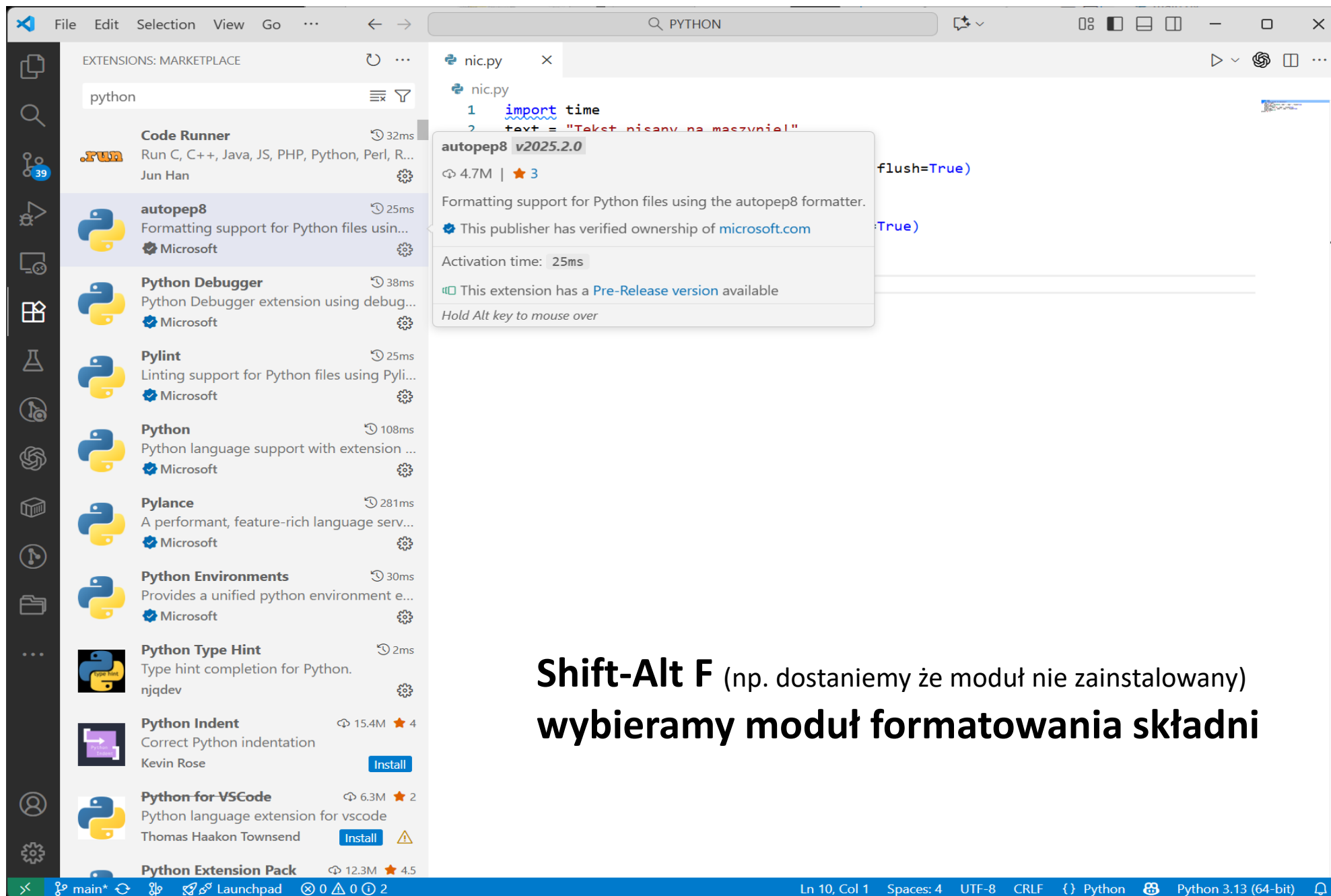
Ctrl ,

PEP – Python Enhancement Proposals <https://peps.python.org/pep-0000/>

Show All Commands Ctrl + Shift + P

terminal u dołu: Ctrl `

COPILOT



Shift-Alt F (np. dostaniemy że moduł nie zainstalowany)
wybieramy moduł formatowania składni

>Python: Select interpreter (na przykład: 3.13.7)

>Preferences: Open User Settings (JSON)

`"python.languageServer": "None"` ← wyłączymy sprawdzanie składni, w tym domyślny linter Pylance

Który linter? <https://code.visualstudio.com/docs/python/linting>

Choose a linter

Search the [VS Code Marketplace](#) for the linter extension of your choice. You can use multiple linters at the same time if you'd like.

Microsoft publishes the following linting extensions for Python:

Linters	Extension
---------	-----------

Pylint	https://marketplace.visualstudio.com/items?itemName=ms-python.pylint
--------	---

flake8	https://marketplace.visualstudio.com/items?itemName=ms-python.flake8
--------	---

mypy	https://marketplace.visualstudio.com/items?itemName=ms-python.mypy-type-checker
------	---

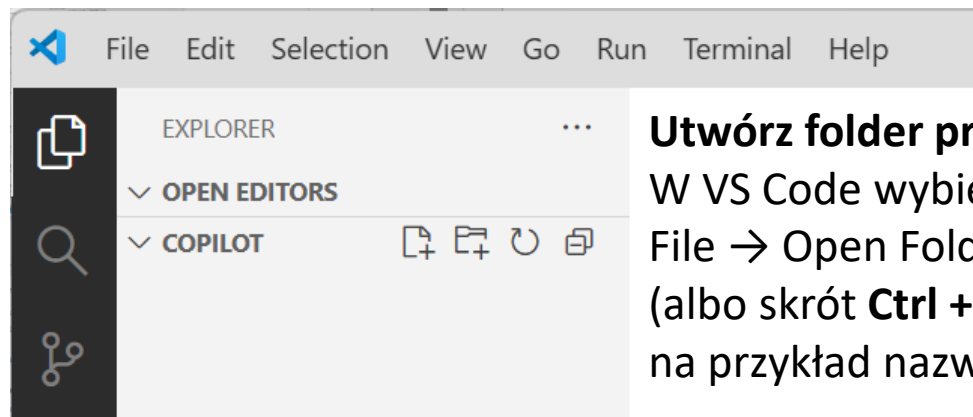
W katalogu domowym plik

.pylintrc

warto ustawić pewne opcje

```
.pylintrc
[BASIC]
module-naming-style=any
[MESSAGES CONTROL]
disable=C0114,C0115,C0116,C0103
```

Utwórzmy w VS Code projekt i prześledźmy kilka kroków z GitHub Copilot

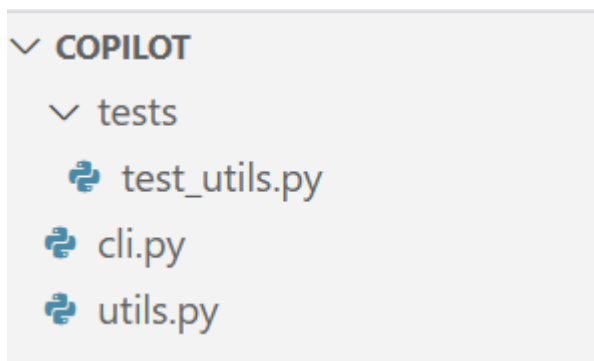
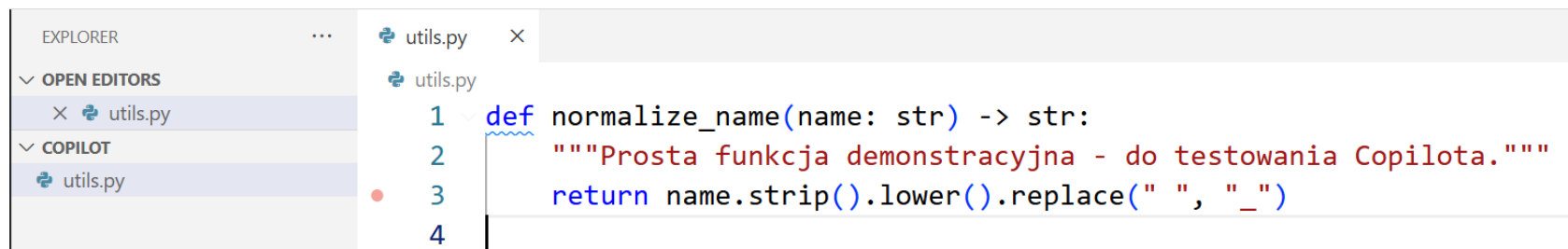
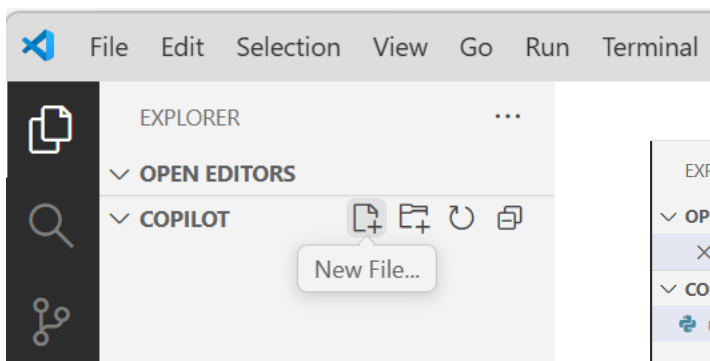


Utwórz folder projektu

W VS Code wybierz z menu:
File → Open Folder...
(albo skrót **Ctrl + K Ctrl + O**)
na przykład nazwa COPILOT

Dodamy teraz kilka plików do nowego projektu, planowana struktura:

```
COPILOT/
├── utils.py
├── cli.py
└── tests/
    └── test_utils.py
```

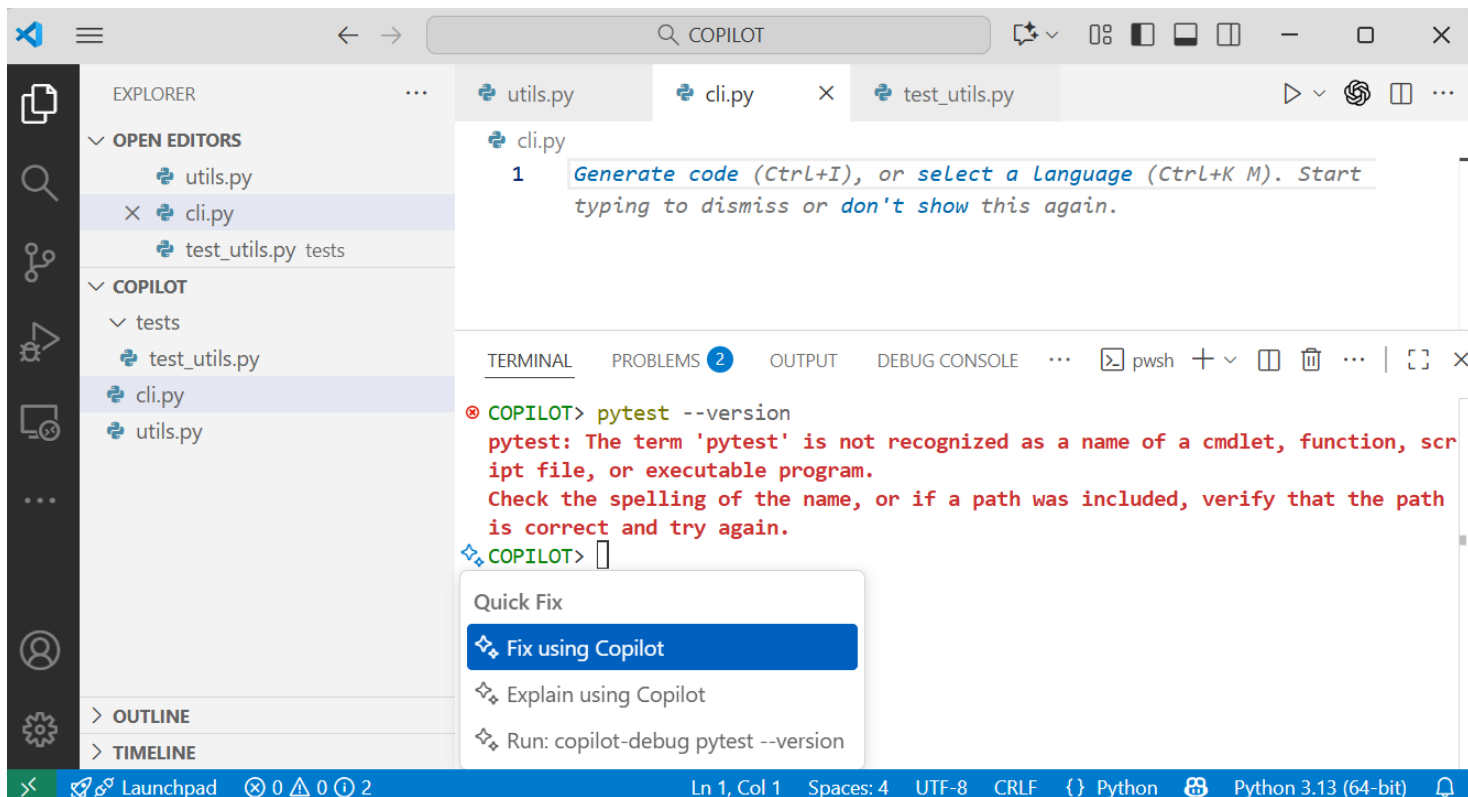


Kolejne pliki mogą być puste:

cli.py

tests/test_utils.py

W terminalu u dołu (**Ctrl + `** lub nowy: **Ctrl + Shift + `**) sprawdzimy, czy mamy zainstalowany pytest i w jakiej wersji:



Jak widać po lewej, nie ma pytest. Możemy go zainstalować sami, ale – ciekawostka – można poprosić o pomoc Copilot – klikamy symbol gwiazdek, wybieramy np. Fix using Copilot.

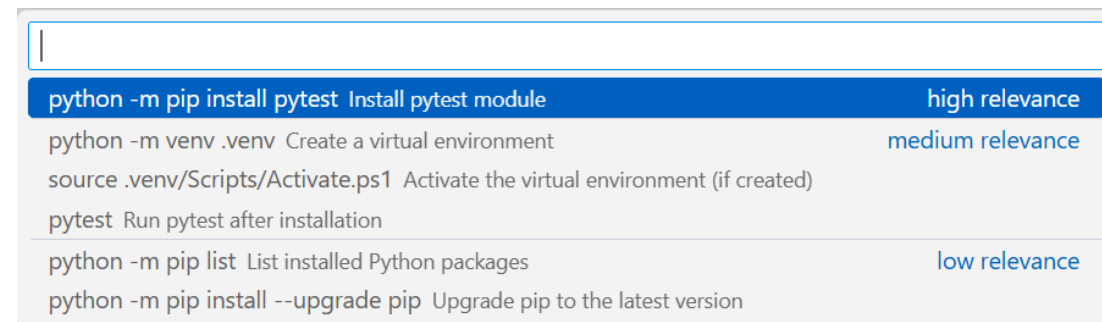
Otrzymamy propozycje – wybieramy pierwszą.

```

COPILOT> python -m pip install pytest
Collecting pytest

```

Sprowadzi się to do wywołania instalacji pytest (oraz pakietów zależnych) za pomocą pip.



Przegląd interaktywnych możliwości GitHub Copilot

Copilot rozumie zamiar z komentarza i nazwy funkcji; mogą przeglądać warianty i wstawić najlepszy jednym Tab.

utils.py

```
1 def normalize_name(name: str) -> str:
2     """Prosta funkcja demonstracyjna - do testowania Copilota."""
3     return name.strip().lower().replace(" ", "_")
```

Quick Fix

Add to Cline

Extract

Explain with Cline

Rewrite

Improve with Cline

Ustawmy kursor przed return.

W zależności od zainstalowanych dodatków (u mnie np. Cline) możemy zobaczyć sugestie interakcji z tymi rozszerzeniami,

Dopiszmy komentarz.

TODO: użyj regexu, zamień ciągi nie-alfanum. na "_"

Pojawi się podpowiedź.

Więcej niż jedno rozszerzenie będzie generować więcej opcji podpowiedzi – chyba warto mieć aktywne jedno rozszerzenie (podobnie jak jeden linter).

```
return re.sub(r'\W+', '_',
```

Quick Fix

Fix with Cline

Add to Cline

Fix

Explain

Extract

Explain with Cline

Rewrite

Improve with Cline

rozszerzenie Codex (od OpenAI) też próbuje zaoferować swoje rozwiązanie...

utils.py

cli.py

test_utils.py

utils.py

```
1 def normalize_name(name: str) -> str:
2     """Prosta funkcja demonstracyjna - do testowania Copilota."""
3     # TODO: użyj regexu, zamień ciągi nie-alfanum. na "_"
4     return name.strip().lower().replace(" ", "_")
5     return re.sub(r'\W+', '_', name.strip().lower())
```

SKRÓTY KŁAWISZOWE

(interaktywnie z Copilot)

Tab – akceptuj

Esc – odrzuć

Alt+] / Alt+[– przeglądaj alternatywy

Alt+ – wymuś nową propozycję

Ctrl+Enter – panel z wieloma propozycjami (wybierz jedną kliknięciem)

Po zatwierdzeniu (Tab) pojawia się problem, moduł re nie jest zaimportowany – re będzie podkreślone na czerwono.

Dalszy scenariusz zależy od naszego wyboru. Np. mogę wybrać **Explain** i Copilot otworzy okno po prawej stronie:

 Workspace

> Used 1 reference

GitHub Copilot

The error means the standard `re` module isn't imported. Add the import at the top. I also suggest a small improvement: replace "non-alphanumeric" (excluding underscores) using `[^A-Za-z0-9]+`, strip outer underscores, and return a fallback for empty input.

```
# ...existing code...
import re

def normalize_name(name: str) -> str:
    """Prosta funkcja demonstracyjna - do testowania Copilota."""
    # zamień ciągi nie-alfanum. na "_", usuń wiodące/końcowe "_" i zwróć pusty
    s = re.sub(r'[^A-Za-z0-9]+', '_', name.strip().lower())
    s = s.strip('_')
    return s or '_'
# ...existing code...
```

Apply in Editor

Gotchas:

- `\w` preserves underscores (because `\w` includes `_`), so I used `[^A-Za-z0-9]` to treat `_` as non-alphanumeric.
- If you want Unicode letters/digits, consider `regex` module or `\p{L}` patterns.

    GPT-5 mini • 0x

Poza właściwą sugestią (import re) pojawi się dodatkowa porada jak można poprawić sam kod.

Możemy zaakceptować poprawkę wprost w naszym kodzie. Poniżej Można scenariusz jeśli wybierzemy **Fix**.

utils.py 2 • cli.py test_utils.py

utils.py

1

 /Fix Undefined variable 're'

The problem is that the 're' module is used but not imported, causing an undefined variable error. To fix this, add an import statement for the 're' module at the top of the file.

Ask or edit in context

Accept Close  

```
2
3 import re
4
5 def normalize_name(name: str) -> str:
6     """Prosta funkcja demonstracyjna - do testowania Copilota."""
7     # TODO: użyj regexu, zamień ciągi nie-alfanum. na "_"
8     return re.sub(r'\W+', '_', name.strip().lower())
9
10
```

Auto	Variable
GPT-4.1	0x
GPT-4o	0x
GPT-5 mini	0x
✓ GPT-5 mini (Preview)	0x
Claude Sonnet 3.5	1x
Claude Sonnet 3.7	1x
Claude Sonnet 3.7 Thinking	1.25x
Claude Sonnet 4	1x
Claude Sonnet 4.5 (Preview)	1x
Gemini 2.0 Flash	0.25x
Gemini 2.5 Pro	1x
GPT-5	1x
GPT-5-Codex (Preview)	1x
o3-mini	0.33x
o4-mini (Preview)	0.33x

Manage Models...

W zależności od modelu otrzymamy proponowane rozwiązanie. Na rozwiniętej liście widzimy modele dostępne w danym momencie. **Należy uważać na to, co proponuje model – bo potrafi zaproponować również błędny kod czy wręcz jakieś absurdalne „śmieci”.**

Praca z kodem. Tryb **Ask**. Zaznaczamy funkcję oraz używamy skrót: **Ctrl+I** i wpisujemy zapytanie.

Wyjaśnij, co robi ta funkcja krok po kroku i zaproponuj testy w pytest.

GPT-5-Codex (Preview) ▾ ▶ ▾

```
def normalize_name(name: str) -> str:
    """Prosta funkcja demonstracyjna - do testowania Copilota."""
```

Implement with Codex

```
    # TODO: użyj regexu, zamień ciągi nie-alfanum. na "_"
    return re.sub(r'\W+', '_', name.strip().lower())
```

Otrzymamy wyjaśnienie, a także propozycję testów.

Jeśli chcemy ich użyć, to najsensowniej w opcjach (ikonki w prawym rogu) skopiować, a potem wkleić do pliku test_utils.py, albo wybrać „Insert into New File” (dalej postępować według uznania). Można też wybrać „View in Chat”, a potem przełączyć na właściwy plik i wybrać opcję Insert At Cursor (skrót Ctrl+Enter).

- Regex `\W+` (w Pythonie: wszystko poza `[A-Za-z0-9_]`) zastępowany jest na `_`, więc wszelkie spacje i znaki specjalne znikają na rzecz podkreśleń.
- Potencjalna pułapka: przed użyciem trzeba `import re`; brak importu zakończy się `NameError`.

Propozycje testów w `pytest` (uwzględniające różne przypadki wejściowe):

```
import pytest
from utils import normalize_name

@pytest.mark.parametrize(
    "raw, expected",
    [
        ("Ala ", "ala"),
        ("Zażółć Gęślą\nJażń", "zażółć_gęślą_jażń"),
        ("user-name@example.com", "user_name_example_com"),
        ("__Already_Clean__", "__already_clean__"),
        ("", ""),
    ],
)
def test_normalize_name_general(raw, expected):
    assert normalize_name(raw) == expected
```

/explain X Wyjaśnij, co robi ta funkcja krok po kroku i zaproponuj testy w pytest.

Funkcja `normalize_name` krok po kroku:

1. Oczekuje napisu `name`.
2. Metoda `strip()` usuwa białe znaki z początku i końca → brak zbędnych odstępów.
3. Metoda `lower()` zamienia litery na małe → ujednolicenie wielkości.
4. Wyrażenie regularne `r'\W+'` w `re.sub` znajduje każdy ciąg znaków niebędących literami, cyframi ani podkreśleniem.
5. Każdy taki ciąg zastępuje pojedynczym `_`, zwracając znormalizowaną nazwę.

Propozycja testów w `pytest`:

```
import re
import pytest
```

(...dalej jest większa porcja kodu testu...)



Ciekawostka: jeśli wywołamy odpowiedź po raz kolejny, to otrzymamy nieco inny tekst i inny przykład testu. Tak manifestuje się działanie wielkich modeli językowych LLM (wybierających odpowiedź na zasadzie probablistyki).

Ask or edit in context

GPT-5-Codex (Preview) ▾ ▶ ▾

Close View in Chat ↻ ▾

Praca z kodem. Tryb **Edit**. Otwieramy panel czatu (**Ctrl+Alt+I**), wybieramy Edit i wpisujemy polecenie.

Zastąp proste replace(" ", "_") regexem `\W+ → _`; dodaj usuwanie diakrytyków przez `unicodedata`; sklej podwójne `_`; dopisz docstring z przykładami i rzuć `TypeError`, jeśli `name` nie jest str.

```

utils.py
1 import re
2 import unicodedata
3
4 def normalize_name(name: str) -> str:
5     """Prosta funkcja demonstracyjna - do testowania Copilota."""
6     # TODO: użyj regexu, zamień ciągi nie-alfanum. na "_"
7     return re.sub(r'\W+', '_', name.strip().lower())
8
9     """Prosta funkcja demonstracyjna - do testowania Copilota.
10
11     Normalizuje nazwę:
12     - wymaga str, w przeciwnym razie wyrzuca TypeError,
13     - usuwa diakrytyki (np. à->a),
14     - zamienia ciągi nie-alfanumeryczne na '_',
15     - scala wielokrotne '_' w jedno i obcina wiodące/końcowe '_'.
16
17     Przykłady:
18     >>> normalize_name("Zażółć gęślą jaźń")
19     'zazolc_gesla_jazn'
20     >>> normalize_name(" Hello, World! ")
21     'hello_world'
22     >>> normalize_name(123)
23     TypeError: name must be str
24     """
25     if not isinstance(name, str):
26         raise TypeError("name must be str")
27
28     # usuń diakrytyki
29     no_diacrit = unicodedata.normalize('NFKD', name)
30     no_diacrit = no_diacrit.encode('ascii', 'ignore').decode('ascii')
31
32     # trim i lowercase
33     s = no_diacrit.strip().lower()
34
35     # zamień ciągi nie-alfanumeryczne na '_'
36     s = re.sub(r'\W+', '_', s)
37
38     # scal wielokrotne '_' w jedno i obetnij brzegowe '_'
39     s = re.sub(r'_{2,}', '_', s).strip('_')
40
41     return s
42
43 test_utils.py
1 import sys
2 from pathlib import Path
3 import pytest
4
5 # zapewnij import z katalogu projektu
6 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
7 from utils import normalize_name
8
9 @pytest.mark.parametrize(
10     "raw, expected",
11     [
12         ("Ala ", "ala"),
13         ("Zażółć Gęślą Jaźń", "zazolc_gesla_jazn"),
14         ("user-name@example.com", "user_name_example_com"),
15         ("Already_Clean ", "already_clean"),
16         ("", ""),
17     ],
18 )
19 def test_normalize_name_general(raw, expected):
20     assert normalize_name(raw) == expected
21
22 @pytest.mark.parametrize("inp, expected", [
23     ("", ""),
24     (" ", " "),
25     ("Zażółć gęślą jaźń", "zazolc_gesla_jazn"),
26     ("a-b.. c", "a_b_c"),
27 ])
28 def test_normalize_name_param(inp, expected):
29     assert normalize_name(inp) == expected
  
```

Spodziewane: lista Changed files + diff w utils.py (import re, unicodedata, walidacja typu, bardziej odporna normalizacja, docstring z doctestami).

Co zrobić: przejrzyj i zaakceptuj diff → Keep. Możemy dopisać w tym samym trybie Edit: Dopisz test parametryzowany w tests/test_utils.py z przypadkami: puste, same spacje, polskie znaki, znaki -, ..

```

test_utils.py
1 import sys
2 from pathlib import Path
3 import pytest
4
5 # zapewnij import z katalogu projektu
6 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
7 from utils import normalize_name
8
9 @pytest.mark.parametrize(
10     "raw, expected",
11     [
12         ("Ala ", "ala"),
13         ("Zażółć Gęślą Jaźń", "zazolc_gesla_jazn"),
14         ("user-name@example.com", "user_name_example_com"),
15         ("Already_Clean ", "already_clean"),
16         ("", ""),
17     ],
18 )
19 def test_normalize_name_general(raw, expected):
20     assert normalize_name(raw) == expected
21
22 @pytest.mark.parametrize("inp, expected", [
23     ("", ""),
24     (" ", " "),
25     ("Zażółć gęślą jaźń", "zazolc_gesla_jazn"),
26     ("a-b.. c", "a_b_c"),
27 ])
28 def test_normalize_name_param(inp, expected):
29     assert normalize_name(inp) == expected
  
```

zaakceptuj

Keep Undo 3 of 3

Praca z kodem. Tryb **Agent**. Otwieramy panel czatu (**Ctrl+Alt+I**), wybieramy Agent i wpisujemy polecenie.

W pliku cli.py utwórz prosty CLI (argparse): csv-tools normalize <INPUT> <OUTPUT> – wczytuje CSV, normalizuje nagłówki kolumn funkcją normalize_name, zapisuje wynik. Dodaj if __name__ == "__main__" i pomoc -h. Na końcu uruchom pytest -q.

The screenshot shows the VS Code editor with three files open: `utils.py`, `cli.py`, and `test_utils.py`. The `cli.py` file contains the following code:

```
1 #!/usr/bin/env python3
2 """Simple CSV tools CLI.
3
4 Usage:
5     csv-tools normalize <INPUT> <OUTPUT>
6
7 Reads a CSV file, normalizes column headers using utils.normalize_name
8 """
9 import argparse
10 import csv
11 import sys
12 from typing import List
13
14 from utils import normalize_name
15
16 def _make_unique(names: List[str]) -> List[str]:
17     """Ensure names are unique by appending _N suffixes to duplicates
18
19     Preserves order.
20     """
21     seen = {}
22     out = []
23     for n in names:
24         if n in seen:
25             seen[n] += 1
26             new = f"{n}_{seen[n]}"
27             # ensure new is also unique
28             while new in seen:
29                 seen[n] += 1
30                 new = f"{n}_{seen[n]}"
31             seen[new] = 0
32             out.append(new)
33         else:
34             seen[n] = 0
35             out.append(n)
36     return out
37
38 def normalize_file(input_path: str, output_path: str):
39     """Read CSV from input_path, normalize headers, write to output_p

The CHAT window on the right shows the following tasks and progress:



- ✓ Created 3 todos
- ✓ Read utils.py
- ✓ Read test_utils.py
- ✓ I'll create cli.py with the requested argparse subcommand and implement normalize_name; expected outcome: new cli.py added. Then I'll u



A red arrow points to the "Allow" button in the chat interface, labeled "zaakceptuj".


```

Spodziewane: Agent pokaże plan, utworzy/zmieni cli.py, zapyta o pozwolenie na zapis/uruchomienie poleceń.

Co zrobić: klikaj **Allow** dla bezpiecznych akcji (np. pytest), przejrzyj diffy; jeśli testy nie przejdą – Agent zaproponuje poprawki, zatwierdź selektywnie.

Agent realizuje cel krokami – edycje wielu plików, a nawet uruchamianie narzędzi, ale **zawsze** z moją zgodą.

The screenshot shows the VS Code editor with the terminal output of the `python -m pytest -q` command. The output shows a failure in the `test_normalize_name_general` test. The chat window on the right shows the following status update:

Status TODO

Zgodnie z planem:

- Odczyt `utils.py` — ukończone
- Implementacja `cli.py` — ukończone
- Uruchomienie testów — ukończone (z 3 nieudanymi testami)

Jeśli chcesz, to: poprawię `utils.normalize_name` tak, aby testy przeszły (albo poprawię testy). Napisz proszę, którą strategię wybieramy — dopasować funkcję do testów, czy testy do funkcji (i czy zachować diakrytyki / wiodące podkreślenia).

At the bottom, there is a button labeled "Keep" and a button labeled "Undo".

Szereg dalszych działań – niekoniecznie zakończonych sukcesem...

Komunikacja z Copilot w okienku czatu.

W Copilot Chat (panel boczny lub inline) można używać trzech rodzajów prefiksów

Symbol @ – dodawanie kontekstu lub narzędzia

Copilot dzięki @ wie, z czego ma czerpać dane do odpowiedzi.

Przykłady:

- @workspace → „Zobacz cały projekt” (analiza struktury repozytorium).
 - „@workspace znajdź wszystkie funkcje, które używają pandas i zaproponuj wspólny moduł.”
 - @file → odnosi się tylko do aktualnie otwartego pliku.
 - „@file opisz, co robi każda funkcja w tym pliku.”
 - @terminal → pozwala generować polecenia powłoki.
 - „@terminal napisz polecenie, które uruchomi pytest dla testów w folderze tests.”
 - @github → pobiera kontekst z repo GitHuba (jeśli repo jest połączzone z kontem).
 - „@github znajdź ostatni commit, który zmienił utils.py.”
- Tip:** możesz łączyć wiele kontekstów, np.
- @workspace @file napisz dokumentację w stylu Google docstring.

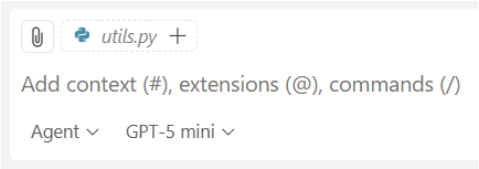
Symbol # – wskazanie konkretnego pliku lub elementu kodu

To precyzyjniejsze „Add context”, zwykle używane **wewnątrz dużych projektów** lub gdy nie chcesz, by Copilot przetwarzał cały workspace.

Przykłady:

- #utils.py → analizuje cały plik.
 - „#utils.py napisz testy do każdej funkcji.”
 - #normalize_name → dodaje kontekst tej funkcji.
 - „#normalize_name zoptymalizuj i dopisz typy wejść.”
- Możesz też wskazać kilka:
- „#utils.py #cli.py porównaj styl funkcji i ujednolicić nazwy.”

Tip: Gdy Copilot nie wie, o jakim pliku lub symbolu mówisz, wpisz #, a VS Code pokaże listę plików i funkcji z Twojego projektu (autocomplete z LSP).



Symbol / – komendy czatu (Copilot Commands)

To **skrótowe polecenia (slash commands)** – działają podobnie jak „prompt templates”. Dzięki nim nie musisz pisać pełnych zdań.

Najczęstsze komendy:

Komenda	Co robi
/explain	Wyjaśnia zaznaczony kod lub aktualny plik
/fix	Szuka błędów, poprawia kod, sugeruje rozwiązanie
/tests	Generuje testy jednostkowe
/optimize	Ulepsza lub skraca kod
/comment	Dodaje komentarze lub docstring
/rename	Proponuje lepsze nazwy funkcji, zmiennych
/terminal	Generuje polecenia bash/powershell do danego celu
/help	Pokazuje dostępne slash-komendy w Twojej wersji Copilota

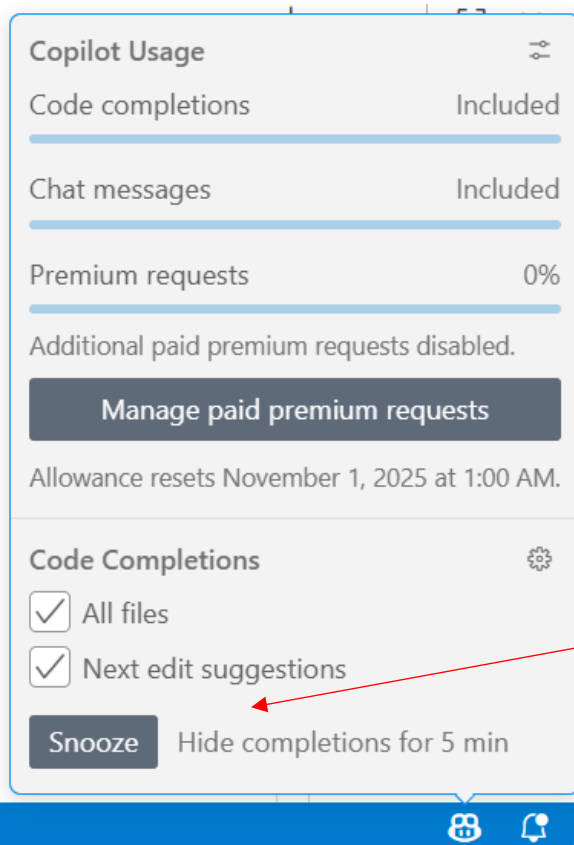
Można też pisać **kombinacje**, np.

- /fix #utils.py → napraw błędy w tym pliku.
- /tests @workspace → napisz testy dla całego projektu.

Praktyczny mini-scenariusz (dla pokazanego przykładu)

Symbol	Użycie	Efekt
@	dodaje kontekst globalny lub narzędzie (workspace, terminal, GitHub)	daje Copilotowi szerszy wgląd
#	wskazuje konkretny plik lub funkcję	precyzyjne polecenia do kodu
/	wywołuje komendę (explain, fix, test, optimize...)	szybkie akcje bez pisania promptów

Jak **wyłączyć** podpowiedzi Copilot (jeśli nas irytują albo mamy więcej rozszerzeń podpowiadających...)

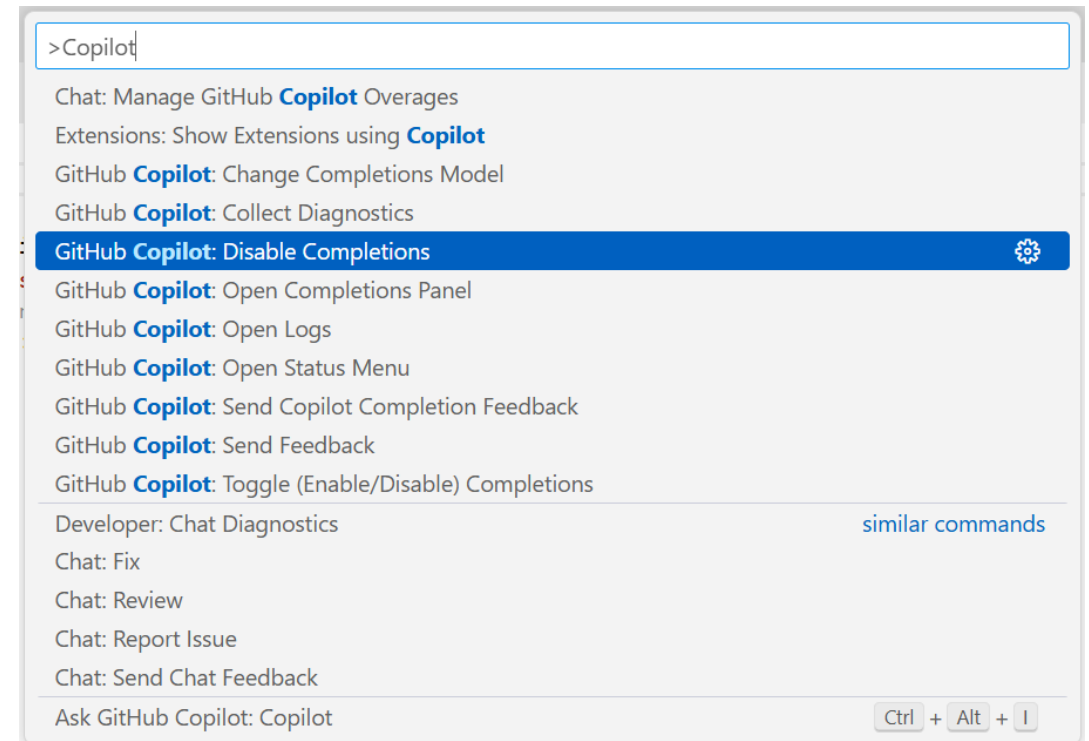


Jeśli chcesz *na chwilę wyciszyć Copilota tylko lokalnie* (np. bo testujesz inne rozszerzenie), wpisz komentarz na górze pliku: `#copilot disable`

Jeśli chcesz wyłączyć na stałe, do ponownego włączenia:

Otwórz **Command Palette** (Ctrl + Shift + P) i wpisz oraz wybierz jak poniżej:

Klikając wielokrotnie w **Snooze** powiększamy odcinek czasu uśpienia. Uwaga: wygląd tego menu bardzo często się zmienia, więc może się okazać, że widzimy coś innego niż na tym slajdzie (stan na październik 2025).



Można też zdefiniować skrót klawiszowy do wyłączania / włączania. Jak to zrobić? Spytaj ChatGPT i uzyskasz szczegółowe informacje.

Podsumowanie. Limity użycia modeli w Copilot Pro.

Na liście jest sporo modeli do wyboru, lista się zmienia w czasie:

Auto	Variable
GPT-4.1	0x
GPT-4o	0x
✓ GPT-5 mini	0x
Grok Code Fast 1 (Preview)	0x
Claude Sonnet 3.5	1x
Claude Sonnet 3.7	1x
Claude Sonnet 4	1x
Claude Sonnet 4.5 (Preview)	1x
Gemini 2.5 Pro	1x
GPT-5	1x
GPT-5-Codex (Preview)	1x
o3-mini	0.33x
o4-mini (Preview)	0.33x
Manage Models...	

W panelu **Copilot Chat** (nie Inline) — w pasku u dołu, obok pola tekstowego — jest **przycisk z nazwą aktualnego modelu**, np. „GPT-5 mini”, „Claude Sonnet 4.5 (Preview)”, „Gemini 2.5 Pro”, „Auto”.

Każdy model ma przypisany **współczynnik zużycia** (mnożnik) określający, jak bardzo dane zapytanie „kosztuje” z Twojej **puli premium** w planie Pro. Modele **x0** są **nielimitowane**, po wykorzystaniu limitu Copilot nadal działa, lecz ogranicza się do modeli **x0**.

Wybór **Auto** (na górze listy) oznacza, że Copilot sam decyduje, który model użyć.

✓ Auto	Variable
GPT-4.1	0x
GPT-4o	0x
GPT-5 mini	0x
Grok Code Fast 1 (Preview)	0x
Claude Haiku 4.5 (Preview)	0.33x
Claude Sonnet 3.5	1x
Claude Sonnet 3.7	1x
Claude Sonnet 3.7 Thinking	1.25x
Claude Sonnet 4	1x
Claude Sonnet 4.5	1x
Gemini 2.0 Flash	0.25x
Gemini 2.5 Pro	1x
GPT-5	1x
GPT-5-Codex (Preview)	1x
o3-mini	0.33x

Ciekawostka: w podręcznym panelu **Ctrl+I** lista modeli jest nieco inna niż w okienku chat

można podejrzeć stopień zużycia

W planie **Copilot Pro** indywidualnie jest **300 premium requests** miesięcznie wliczone w plan. Pełna charakterystyka: <https://docs.github.com/en/copilot/get-started/plans>

	Copilot Free	Copilot Pro
Pricing	Not applicable	\$10 USD per month, or \$100 USD per year (free for some users)
Premium requests	50 per month	300 per month
Purchase additional premium requests at \$0.04/request	×	✓

Mnożnik	Znaczenie / koszt	Dostępne modele	Typowe zastosowanie
x0	◆ Bezpłatne (nie zużywają limitu premium)	GPT-4.1GPT-4oGPT-5 miniGrok Code Fast 1 (Preview)	Błyskawiczne odpowiedzi inline, wyjaśnienia kodu, proste Q&A, krótkie refaktoryzacje
x0.33	◆ Lekkie modele – 3 takie zapytania ≈ 1 premium request	o3-minio4-mini (Preview)	Szybkie generacje, testy jednostkowe, krótkie polecenia
x1	◆ Pełny koszt 1 premium request	Claude Sonnet 3.5 / 3.7 / 4 / 4.5 (Preview); Gemini 2.5 ProGPT-5GPT-5 Codex (Preview)	Dłuższe analizy, praca z wieloma plikami, tworzenie struktur projektowych, generacja większych fragmentów kodu

Copilot Usage	
Code completions	Included
Chat messages	Included
Premium requests	1.7%
Additional paid premium requests disabled.	
Manage paid premium requests	
Allowance resets November 1, 2025 at 1:00 AM.	
Chat Sessions	
Local Chat Sessions in progress	
Code Completions	
All files	
Python	
Next edit suggestions	
Snooze	
Hide completions for 5 min	

- „Preview” to wersja beta udostępniona przez GitHub.
- „Manage Models” pozwala dodać lub ograniczyć modele, a w wersji Pro można dodać inny model przez klucz API (np. z OpenAI Pro lub Anthropic API) – czyli zwykle płacąc za zużycie (rozliczenie po stronie modelu).